

✓ BPL_TEST2_Batch script with OMSimulator

The key library OMSimulator is installed.

After the installation a small application BPL_TEST2_Batch is loaded and run. You can continue with this example if you like.

```
!lsb_release -a # Actual VM Ubuntu version used by Google
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 24.04.5 LTS
Release:      24.04
Codename:     noble
```

```
!python --version
```

```
Python 3.13.15
```

```
!pip install OMSimulator
```

```
Collecting OMSimulator
  Downloading omsimulator-3.0.0.post116.tar.gz (7.6 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (2.32.4)
Requirement already satisfied: lxml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (6.1.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests->OMSimulator) (2025.11.12)
Building wheels for collected packages: OMSimulator
  Building wheel for OMSimulator (pyproject.toml) ... done
  Created wheel for OMSimulator: filename=omsimulator-3.0.0.post116-py3-none-any.whl size=1345325 sha256=28d96a2e5b0ee5194adbcfe75a2efc0262c5c0377b28ecbc648628be707b5a4
  Stored in directory: /root/.cache/pip/wheels/eb/b0/ee/5194adbcfe75a2efc0262c5c0377b28ecbc648628be707b5a4
Successfully built OMSimulator
Installing collected packages: OMSimulator
Successfully installed OMSimulator-3.0.0.post116
```

✓ BPL_TEST2_Batch setup

Now specific installation and the run simulations. Start with connecting to Github. Then upload the two files:

- FMU - BPL_TEST2_Batch_linux_om_me.fmu
- Setup-file - BPL_TEST2_Batch_fmpy_explore.py

```
%%bash
git clone https://github.com/janpeter19/BPL_TEST2_Batch
```

```
Cloning into 'BPL_TEST2_Batch'...
```

```
%%cd BPL_TEST2_Batch
```

```
/content/BPL_TEST2_Batch
```

✓ BPL_TEST2_Batch - demo

```
import OMSimulator as oms
```

```
import scipy.io
```

```
import matplotlib.pyplot as plt
```

```
fmu = oms.FMU('BPL_TEST2_Batch_linux_om_me.fmu')
```

```
fmu.modelName
```

```
'BPL.Examples_TEST2.Batch'
```

```
for s in fmu.states:
    print(s.name)
```

```
bioreactor.V
bioreactor.m[1]
bioreactor.m[2]
```

```
help(fmu)
```

Help on FMU in module OMSimulator.fmu object:

```
class FMU(builtins.object)
|   FMU(fmu_path: Union[str, pathlib._local.Path], instanceName: str = None)
|
|   Methods defined here:
|
|   __init__(
|       self,
|       fmu_path: Union[str, pathlib._local.Path],
|       instanceName: str = None
|   )
|       Initialize the FMU by loading modelDescription.xml from the FMU archive.
|
|   delete(self)
|
|   dumpApiCalls(self)
|       Returns the generated API calls as a string.
|
|   exportSSMTemplate(self, filename: pathlib._local.Path | None = None)
|
|   exportSSVTemplate(self, filename: pathlib._local.Path | None = None)
|
|   export_units_to_ssd(self, node)
|       Exports all unit definitions to an SSD XML node.
|
|   getValue(self, cref: str)
|
|   initialize(self)
|
|   instantiate(self)
|       Instantiate the FMU for simulation.
|
|   makeConnectors(self)
|
|   setDefaultExperiment(self)
|
|   setLogFile(self, filename: str)
|
|   setLoggingInterval(self, interval: float)
|
|   setLoggingLevel(self, level: int)
|
|   setResultFile(self, filename: str)
|
|   setStartTime(self, startTime: float)
|
|   setStepSize(self, stepSize: float)
|
|   setStopTime(self, stopTime: float)
|
|   setTolerance(self, tolerance: float)
|
|   setValue(self, cref: str, value)
|       Sets a value for a specific CRef in the model.
|
|   simulate(self)
```

```
fmu.instantiate()
```

```
fmu.setResultFile('result.mat')
```

```
fmu.setValue('bioreactor.V', 1.0)
fmu.setValue('bioreactor.m_start[1]', 1.0)
fmu.setValue('bioreactor.m_start[2]', 10.0)
```

```

RuntimeError                                Traceback (most recent call last)
/tmp/ipykernel_1573/1766152030.py in <cell line: 0>()
----> 1 fmu.setValue('bioreactor.V', 1.0)
      2 fmu.setValue('bioreactor.m_start[1]', 1.0)
      3 fmu.setValue('bioreactor.m_start[2]', 10.0)

1 frames
/usr/local/lib/python3.13/dist-packages/OMSimulator/fmu.py in _setReal(self, mapped_cref, value)
   570     status = Capi.setReal(mapped_cref, value) # Get the value from the CAPI
   571     if status != Status.ok:
--> 572         raise RuntimeError(f"Failed to set value for {mapped_cref}: {status}")
   573
   574     def _setInteger(self, mapped_cref: str, value: int):

RuntimeError: Failed to set value for Batch.root.Batch.bioreactor.V: Status.warning

```

Next steps: [Explain error](#)

```
fmu.initialize()
```

```
fmu.simulate()
```

```
mat_data = scipy.io.loadmat('result.mat')
```

```
mat_data.keys()
```

```
dict_keys(['Aclass', 'name', 'description', 'dataInfo', 'data_1', 'data_2'])
```

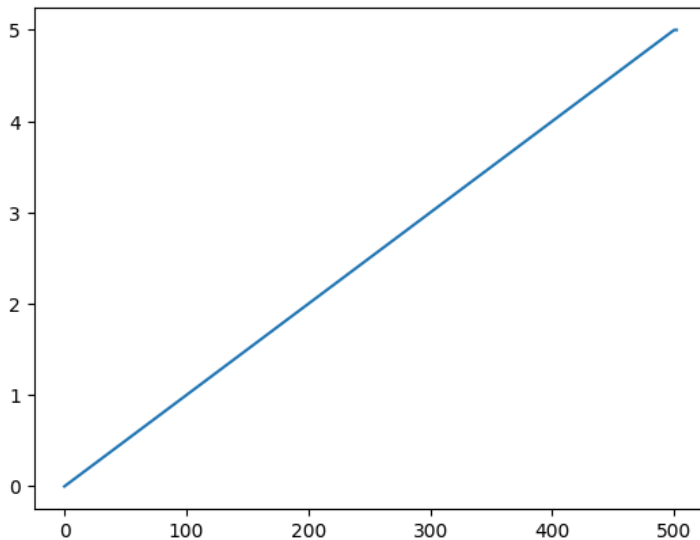
```
mat_data['Aclass']
```

```
array(['Atrajjectory', '1.1', '', 'binTrans'], dtype='<U11')
```

```
t = mat_data['data_2'][0];
```

```
plt.figure()
plt.plot(mat_data['data_2'][0])
```

```
[<matplotlib.lines.Line2D at 0x793b170b4e10>]
```



Simulation time is apparently store in "0". The other "columns" show just constants. I think the initial values of the states need to be set different from zero which is default.

Start coding or [generate](#) with AI.

